Gestionnaire de tâches avec React

Corrigé pédagogique en français • Projet réalisé progressivement • Sans logo

Ce document reprend les étapes 0 à 9 du sujet « Projet React débutant - Gestionnaire de tâches » (7 pages) et répond à toutes les questions « À expliquer ». Il décrit le projet JavaScript réellement présent dans src/App.jsx, ainsi que son API FastAPI dans api/main.py.

Ce que le projet permet de comprendre

Une action dans le navigateur déclenche une requête HTTP. Uvicorn la reçoit et la transmet à FastAPI. FastAPI traite les données et renvoie une réponse JSON. React met alors son état à jour et actualise la page.

```
React → fetch → HTTP → Uvicorn → FastAPI
React ← response.json() ← réponse HTTP / JSON ← FastAPI
setTasks(data) → nouvel affichage
```

Comment lire les preuves

Les captures sont celles fournies pendant les essais. Elles représentent des moments différents du projet : la première interface ne comporte donc pas encore les boutons de suppression. Une capture commune peut illustrer plusieurs notions, sans prouver à elle seule chaque interaction.

Les quatre opérations de l’API ont été testées dans la documentation interactive. Le cycle final React a ensuite été vérifié dans le navigateur : création, rechargement, changement de statut, filtre et suppression. Quatre nouvelles captures documentent ces essais. Les contrôles de code, de compilation et d’API sont également passés sur GitHub.

Une différence avec la consigne

Le sujet initial utilise TypeScript et App.tsx. Le projet utilise JavaScript et App.jsx avec des commentaires JSDoc. Le bon état initial est donc useState([]), sans la syntaxe TypeScript <Task[]>.

Organisation du corrigé

Préparation et données • Composants et état • Formulaire • Statut et filtre • Effets • API • Chargement HTTP • CRUD • Mémoire et serveur • Code final en annexe

0 Préparer le projet

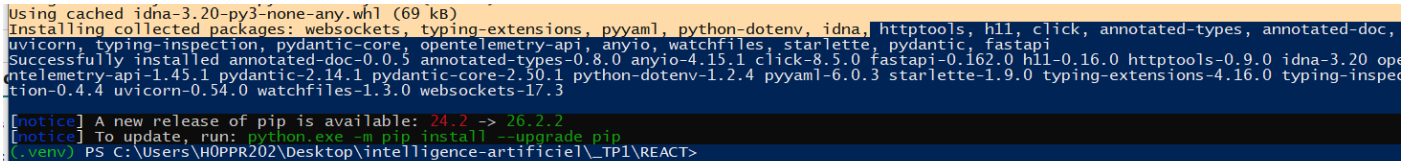
Fichiers concernés : package.json, src/main.jsx, src/App.jsx et environnement Python

Vite lance le serveur de développement de la page React. Uvicorn lance séparément le serveur de l'API. Les deux terminaux doivent rester actifs pendant les essais.

```
cd "C:\Users\H0PPR202\Desktop\intelligence-artificiel\_TP1\REACT\react-tasks"
npm.cmd install
npm.cmd run dev
```

Dans cette réalisation, .venv a été créé dans le dossier parent REACT. Pour démarrer l'API dans un deuxième terminal :

```
cd "C:\Users\H0PPR202\Desktop\intelligence-artificiel\_TP1\REACT"
.\.venv\Scripts\Activate.ps1
cd .\react-tasks
python -m uvicorn api.main:app --reload
```



Capture fournie : installation réussie de FastAPI et Uvicorn dans .venv. La commande d'installation est python -m pip install fastapi "uvicorn[standard]".

Réponse à retenir

Un serveur est un programme qui reçoit des demandes et renvoie des réponses. Uvicorn est un véritable serveur HTTP. FastAPI définit les routes et les traitements de l'API. Le navigateur affiche React.

1 Comprendre les données

Fichier : javascript-basics.js

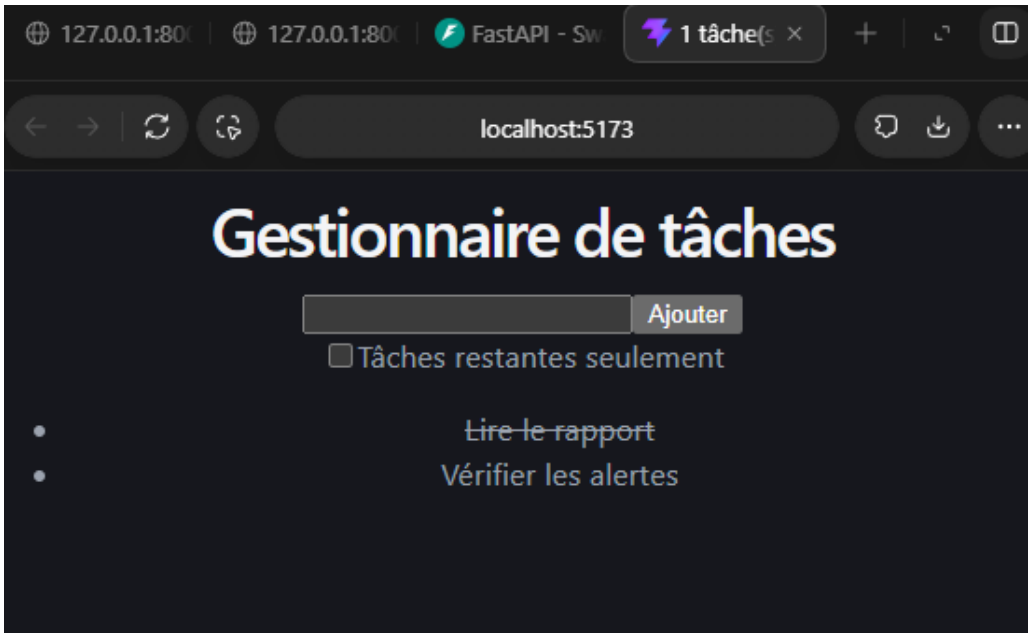
```
const tasks = [
  { id: 1, title: "Lire le rapport", done: true },
  { id: 2, title: "Vérifier les alertes", done: false },
];
console.log(tasks[0].title);
const remainingTasks = tasks.filter((task) => !task.done);
console.log(remainingTasks.length);
```

Réponse expliquée

Un tableau est une collection ordonnée : ici, tasks contient plusieurs tâches. Un objet regroupe des propriétés nommées : chaque tâche possède id, title et done. tasks[0] est le premier objet du tableau et tasks[0].title est son titre.

filter conserve les objets pour lesquels la condition est vraie. !task.done signifie « la tâche n'est pas terminée ». Le résultat attendu est Lire le rapport, puis 1.

Vérification proposée : exécuter node javascript-basics.js depuis la racine du projet. Aucune capture de cette exécution n'a été fournie ; les résultats ci-dessus sont calculés à partir du code, et non présentés comme une sortie capturée.



Capture commune de l'interface : les objets sont affichés sous forme de tâches. Elle illustre le modèle de données, sans remplacer le test de javascript-basics.js.

2 et 3 Composants props et état

Fichier : src/App.jsx

Un composant

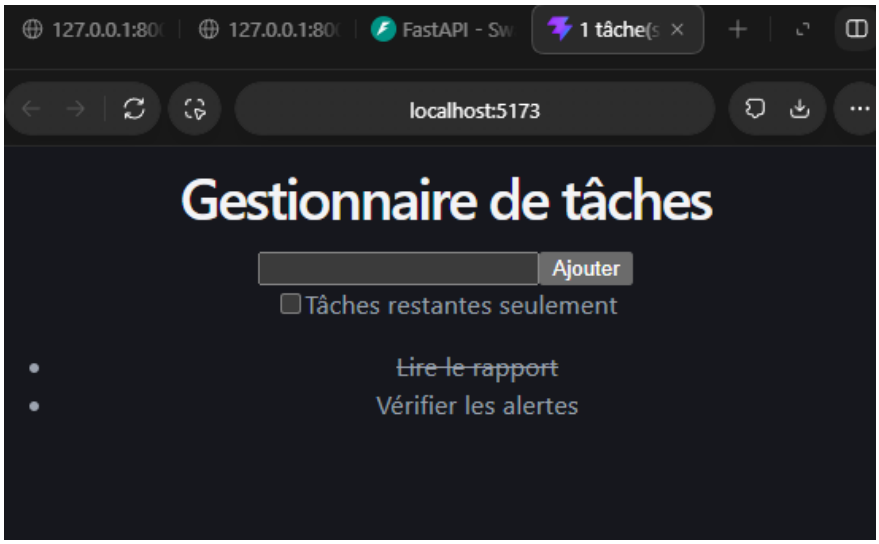
Un composant est une fonction qui décrit une partie de l'interface en JSX. TaskHeader retourne le titre. App assemble les composants dans main. La majuscule permet à React de distinguer un composant, comme <TaskHeader />, d'un élément HTML, comme <h1>.

```
function TaskHeader() {
  return <h1>Gestionnaire de tâches</h1>
}
// Dans App :
const [tasks, setTasks] = useState([])
// Dans la liste :
<TaskItem key={task.id} task={task} onToggle={toggleTask}
  onDelete={deleteTask} />
```

Une prop et un état

Une prop vient du parent. App fournit task à TaskItem. L'enfant la lit et reçoit également des fonctions pour signaler une action. useState conserve une valeur entre les rendus ; son setter demande un nouveau rendu. App possède tasks et TaskForm possède title.

Au début du parcours, tasks contenait deux tâches dans React. Depuis l'étape 8, tasks commence vide, puis reçoit les données de l'API. Le type Task du sujet devient un commentaire JSDoc dans ce projet JavaScript.



Capture commune des étapes 2 à 6 : le titre et la liste montrent les composants rendus. Le code explique la circulation des props.

4 Ajouter une tâche

Composants : TaskForm et App

Le champ est contrôlé : sa valeur vient de title. onChange copie chaque nouvelle saisie dans cet état. onSubmit appelle handleSubmit lorsque le formulaire est validé, y compris avec la touche Entrée.

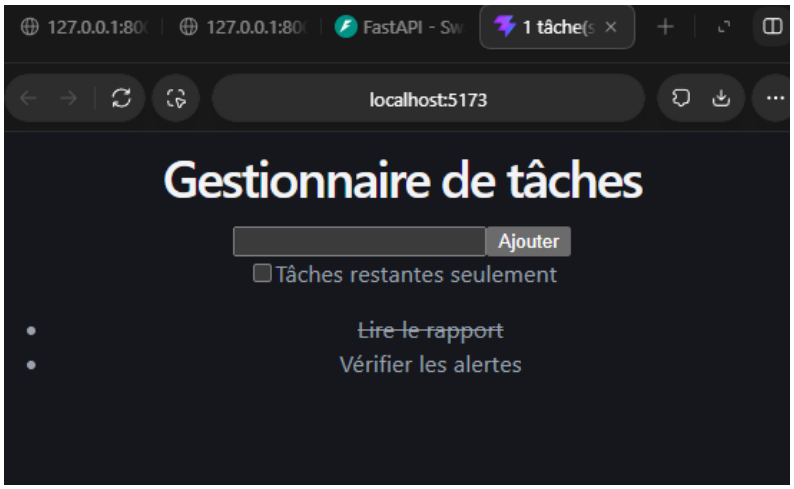
```
const [title, setTitle] = useState('')
<input value={title}
  onChange={(event) => setTitle(event.target.value)} />

function handleSubmit(event) {
  event.preventDefault()
  const trimmedTitle = title.trim()
  if (!trimmedTitle) return
  onAdd(trimmedTitle)
  setTitle('')
}
```

Réponses à expliquer

onChange réagit à la saisie. onSubmit réagit à la validation du formulaire. event.preventDefault() empêche le comportement HTML habituel qui rechargerait la page. trim enlève les espaces au début et à la fin ; le test suivant refuse un titre vide.

Flux : saisie → setTitle → validation → onAdd → addTask dans App. À l'étape 4, addTask ajoutait un objet local avec Date.now(). Dans la version finale, il envoie POST et utilise l'objet créé par le serveur.



Capture commune : le champ et le bouton Ajouter sont visibles. La création finale avec POST est expliquée à l'étape 9.

Limite du code actuel : le champ est vidé dès l'appel à onAdd, même si la requête échoue ensuite. La liste tasks, elle, n'est modifiée qu'après une réponse réussie.

5 Terminer et filtrer les tâches

États : tasks et remainingOnly

```
const visibleTasks = remainingOnly
  ? tasks.filter((task) => !task.done)
  : tasks

currentTasks.map((t) => t.id === taskId ? updated : t)

<TaskItem key={task.id} task={task}
  onToggle={toggleTask} onDelete={deleteTask} />
```

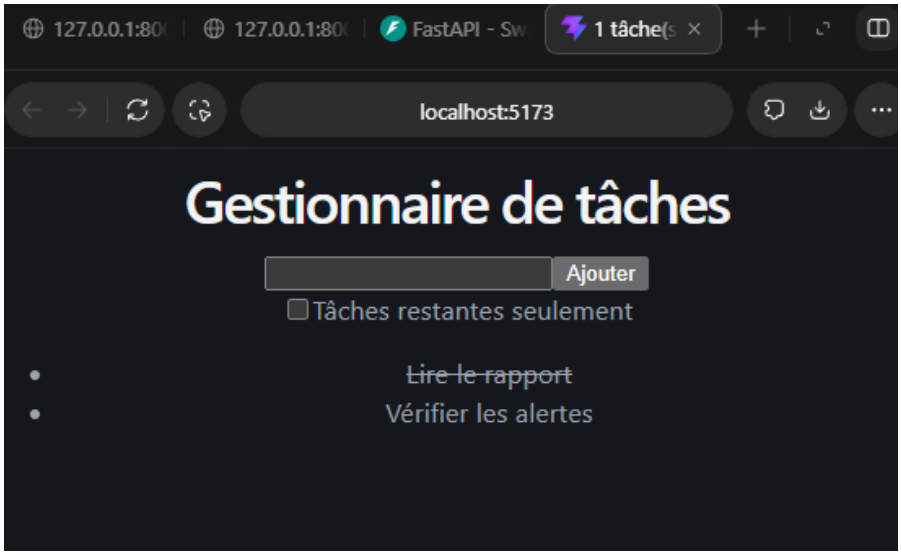
map et filter

map produit un résultat pour chaque élément : il sert à créer les éléments JSX, ou à remplacer la tâche modifiée dans une nouvelle liste. filter conserve seulement les éléments qui satisfont une condition : il sélectionne les tâches restantes.

Cocher le filtre ne supprime aucune tâche. Le code calcule visibleTasks à partir de tasks. Si la liste visible est vide, React affiche « Aucune tâche ».

Identité et changement de statut

key={task.id} donne à React une identité stable pour chaque élément entre les rendus. Un changement de statut transmet task.id au parent. Dans la version finale, PATCH modifie done sur le serveur, puis map remplace la tâche dans l'état React.



Capture commune : Lire le rapport est barré et Vérifier les alertes ne l'est pas. La case de filtre est visible. Cette image ne montre pas encore les boutons de la version finale.

6 Synchroniser le titre de l’onglet

Hook : useEffect

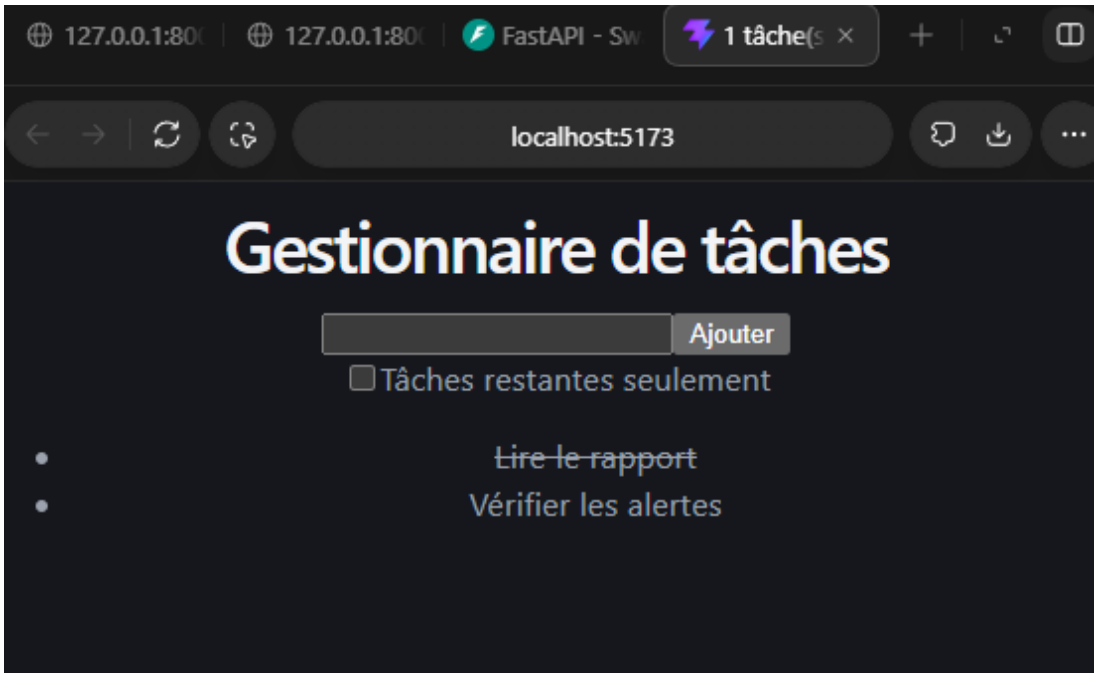
```
const remainingCount = tasks.filter((task) => !task.done).length

useEffect(() => {
  document.title = `${remainingCount} tâche(s) restante(s)`
}, [remainingCount])
```

Pourquoi utiliser un effet

document.title appartient au document du navigateur, en dehors du JSX retourné par le composant. useEffect sert ici à synchroniser ce titre avec les données React après le rendu. L’effet s’exécute au montage et lorsque remainingCount change.

Le tableau [remainingCount] indique la dépendance. Un nouveau rendu où ce nombre reste identique ne demande pas de répéter cet effet. En développement, StrictMode peut provoquer un cycle supplémentaire de vérification des effets.



Capture : le titre de l’onglet commence par « 1 tâche(s) », ce qui correspond à la seule tâche non terminée affichée.

Flux à expliquer

tasks → sélection des tâches non terminées → nombre remainingCount → useEffect → document.title. Le filtre d’affichage ne change pas ce compteur, car il est calculé à partir de toutes les tâches.

7 Créer et tester la petite API

Fichier : api/main.py

```
app = FastAPI()

@app.get("/tasks")
def get_tasks():
    return tasks
```

React et FastAPI fonctionnent dans deux programmes distincts. Dans ce TP, ils tournent sur le même ordinateur. React est ouvert sur localhost:5173 ; Uvicorn écoute l'API sur 127.0.0.1:8000. CORS autorise le navigateur à lire les réponses de l'API depuis l'origine http://localhost:5173.



Capture GET : l'exécution de /tasks renvoie le code 200 et les deux objets JSON. 200 signifie que la requête a réussi.

Adresse et route

127.0.0.1 désigne cet ordinateur, 8000 est le port et /tasks est la route demandée. /docs affiche la documentation interactive. La route / n'étant pas définie, elle renvoie Not Found ; cela ne signifie pas que le serveur est arrêté.

8 Charger les tâches depuis l’API

Fichier : src/App.jsx

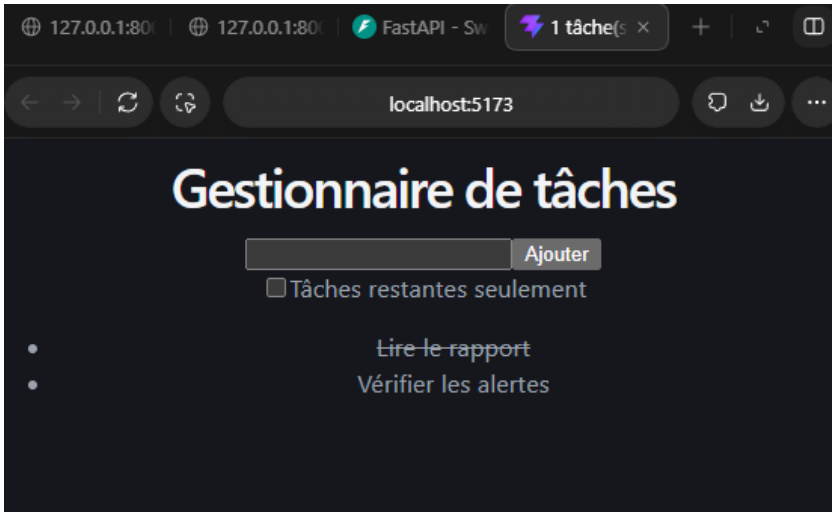
```
const API_URL = 'http://127.0.0.1:8000/tasks'
const [tasks, setTasks] = useState([])
const [loading, setLoading] = useState(true)
const [error, setError] = useState('')

// Dans la fonction asynchrone appelée par useEffect :
const response = await fetch(API_URL)
if (!response.ok) throw new Error(`Erreur HTTP ${response.status}`)
const data = await response.json()
setTasks(data)
```

HTTP JSON et await

HTTP est le protocole des échanges entre client et serveur. JSON est un format textuel de données. response.json() lit le corps de la réponse et le transforme en données JavaScript. await suspend la suite de la fonction asynchrone jusqu’au résultat, sans bloquer l’interface.

fetch ne rejette pas automatiquement une réponse HTTP 404 ou 500 : response.ok doit être contrôlé. Le catch renseigne error et le finally termine loading. Le JSX affiche Chargement..., une erreur, ou la liste. Le code complet en annexe évite aussi les mises à jour d’un effet devenu inactif avec cancelled.



Capture de l’étape 8 : les deux tâches sont affichées après le chargement. Le formulaire et le filtre sont conservés.

9 Créer une tâche avec POST

API et React

```
const created = await request(API_URL, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ title }),
})
if (created) setTasks((currentTasks) => [...currentTasks, created])
```

JSON.stringify transforme l'objet JavaScript en texte JSON pour l'envoi. FastAPI valide le titre avec TaskCreate, construit un identifiant et retourne la tâche créée avec le statut 201. React ajoute cet objet à sa liste après réussite.



Capture POST fournie : Préparer le cours a été créé avec id 3 et done false. Le code 201 confirme la création.

Validation : POST a été testé dans /docs puis depuis le formulaire React. La tâche de validation reste visible après un rechargement du navigateur, ce qui confirme son retour depuis l'API. Les captures du cycle React figurent après les essais DELETE.

9 Modifier une tâche avec PATCH

Route : /tasks/{task_id}

```
const updated = await request(`${API_URL}/${taskId}`, {
  method: 'PATCH',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ done: !task.done }),
})
if (updated) {
  setTasks((currentTasks) =>
    currentTasks.map((t) => t.id === taskId ? updated : t))
}
```

task_id identifie la tâche à modifier. Le corps contient seulement les propriétés à changer. Le serveur utilise model_dump(exclude_unset=True) pour conserver les propriétés non envoyées, notamment le titre.



Capture PATCH : la tâche 3 conserve son titre et passe à done true. Le code 200 confirme la modification.

Dans TaskItem, le bouton Changer le statut appelle onToggle(task.id). App transmet toggleTask comme callback. Le texte est barré lorsque task.done est vrai. L'utilisateur a également signalé que ce changement fonctionne dans l'interface React.

9 Supprimer une tâche avec DELETE

Route : /tasks/{task_id}

```
const result = await request(`${API_URL}/${taskId}`, {
  method: 'DELETE',
})
if (result) {
  setTasks((currentTasks) =>
    currentTasks.filter((t) => t.id !== taskId))
}
```

FastAPI retrouve la tâche, la retire de la liste Python et retourne {"deleted": task_id}. Après réussite, filter construit la liste React sans cet identifiant. Le bouton Supprimer et le bouton Changer le statut sont placés dans le même li, sans être imbriqués.

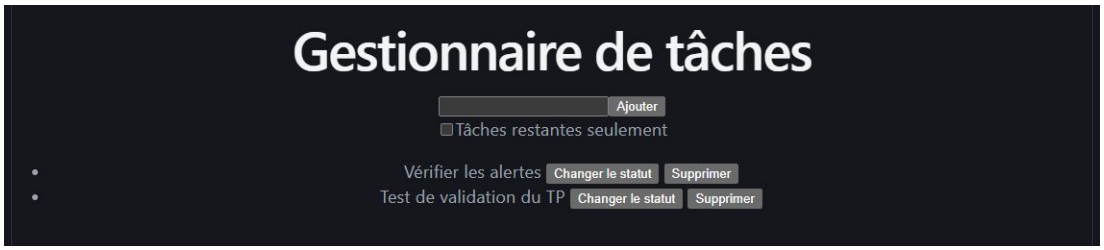


Capture DELETE : le serveur répond 200 avec deleted égal à 3. La tâche de test a été supprimée.

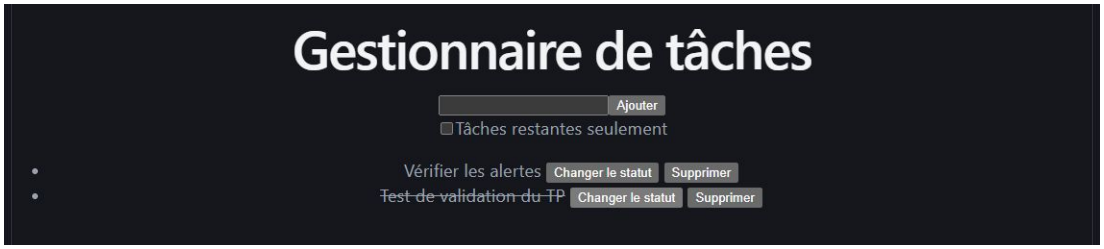
La fonction request vérifie response.ok et affiche actionError en cas d'échec. Elle retourne null en cas d'erreur ; les fonctions d'action n'appliquent alors aucun changement à tasks. L'utilisateur a signalé une suppression réussie depuis React.

9 Vérifier les actions dans React

Essais réalisés dans le navigateur le 24 octobre 2026. Une seule tâche temporaire a été créée puis supprimée ; la tâche existante Vérifier les alertes a été conservée.



POST depuis React : la tâche de validation reste présente après rechargement.



PATCH depuis React : la tâche est barrée et le compteur diminue.



Filtre coché : la tâche terminée est masquée, sans être supprimée.



DELETE puis rechargement : la tâche de validation est absente.

Serveur mémoire et limites du projet

Où se trouve le serveur

Uvicorn est installé dans le dossier .venv du projet parent. Son exécutable existe sur le disque ; le serveur en activité est un processus qui utilise la RAM du PC. Il n'a pas de bureau graphique. /docs est une interface de test de l'API fournie par FastAPI.

La case « Impression élégante » du navigateur sert simplement à présenter le JSON avec des retraits et des retours à la ligne. Elle ne lance aucune impression et ne modifie ni les tâches ni le serveur.

Ce qui reste après un rechargement

Recharger React relance le chargement GET : les données encore présentes dans la liste Python reviennent. En revanche, arrêter ou redémarrer le processus de l'API réinitialise cette liste à partir du fichier main.py. --reload peut provoquer ce redémarrage après une modification du code.

Une base de données permettrait de conserver les changements durablement. Elle n'est pas utilisée dans ce TP. Uvicorn reste un véritable serveur même lorsque le stockage est une simple liste en mémoire.

Limites de cette version pédagogique

L'API n'a pas d'authentification. Elle ne rejette pas encore les titres composés uniquement d'espaces côté serveur. TaskUpdate accepte aussi null, même si l'interface normale envoie un booléen pour done. Les requêtes simultanées et les clics répétés ne sont pas gérés comme dans une application complète. Ces points ne bloquent pas les essais demandés, mais doivent être améliorés pour un usage plus large.

Bilan des vérifications

GET : capture 200 avec la liste. POST : capture 201 avec la tâche 3. PATCH : capture 200 avec done true. DELETE : capture 200 avec deleted 3. React : création suivie d'un rechargement, changement de statut, filtre et suppression suivie d'un rechargement vérifiés et capturés. Les tests isolés HTTP et les contrôles GitHub ont également réussi.

Explication orale proposée

« React est le client. Quand j'effectue une action, il envoie une requête HTTP à l'API. Uvicorn reçoit cette requête et FastAPI exécute la route correspondante. Le serveur renvoie du JSON. Après une réponse réussie, React met son état à jour et actualise la page. Les données du serveur restent ici en mémoire et ne survivent pas à un redémarrage. »

Annexe du code final

javascript-basics.js • partie 1 sur 1 • Copie du fichier relu

```
const tasks = [
  { id: 1, title: "Lire le rapport", done: true },
  { id: 2, title: "Vérifier les alertes", done: false },
];

console.log(tasks[0].title);

const remainingTasks = tasks.filter((task) => !task.done);
console.log(remainingTasks.length);
```

Annexe du code final

api/main.py • partie 1 sur 2 • Copie du fichier relu

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_methods=["*"],
    allow_headers=["*"],
)

tasks = [
    {"id": 1, "title": "Lire le rapport", "done": True},
    {"id": 2, "title": "Vérifier les alertes", "done": False},
]


@app.get("/tasks")
def get_tasks():
    return tasks


class TaskCreate(BaseModel):
    title: str


class TaskUpdate(BaseModel):
    title: str | None = None
    done: bool | None = None


def find_task(task_id: int):
    for task in tasks:
        if task["id"] == task_id:
            return task
    raise HTTPException(status_code=404, detail="Tâche introuvable")


@app.post("/tasks", status_code=201)
def create_task(data: TaskCreate):
    task = {"id": max((t["id"] for t in tasks), default=0) + 1, "title": data.title, "done": False}
    tasks.append(task)
    return task


@app.patch("/tasks/{task_id}")
def update_task(task_id: int, data: TaskUpdate):
```

Annexe du code final

api/main.py • partie 2 sur 2 • Copie du fichier relu

```
50     task = find_task(task_id)
51     task.update(data.model_dump(exclude_unset=True))
52     return task
53
54
55 @app.delete("/tasks/{task_id}")
56 def delete_task(task_id: int):
57     task = find_task(task_id)
58     tasks.remove(task)
59     return {"deleted": task_id}
```

Annexe du code final

src/App.jsx • partie 1 sur 4 • Copie du fichier relu

```jsx
import { useEffect, useState } from 'react'

/**
 * @typedef {Object} Task
 * @property {number} id
 * @property {string} title
 * @property {boolean} done
 */

function TaskHeader() {
  return <h1>Gestionnaire de tâches</h1>
}

/** @param {{ task: Task, onToggle: (taskId: number) => void, onDelete: (taskId: number) => void }} props */
function TaskItem({ task, onToggle, onDelete }) {
  return (
    <li>
      <span
        style={{ textDecoration: task.done ? 'line-through' : 'none' }}
      >
        {task.title}
      </span>{' '}
      <button type="button" onClick={() => onToggle(task.id)}>
        Changer le statut
      </button>{' '}
      <button type="button" onClick={() => onDelete(task.id)}>
        Supprimer
      </button>
    </li>
  )
}
/** @param {{ onAdd: (title: string) => void }} props */
function TaskForm({ onAdd }) {
  const [title, setTitle] = useState('')

  /** @param {import('react').FormEvent<HTMLFormElement>} event */
  function handleSubmit(event) {
    event.preventDefault()
    const trimmedTitle = title.trim()
    if (!trimmedTitle) return

    onAdd(trimmedTitle)
    setTitle('')
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
```

Annexe du code final

src/App.jsx • partie 2 sur 4 • Copie du fichier relu

```
        type="text"
        value={title}
        onChange={(event) => setTitle(event.target.value)}
      />
      <button type="submit">Ajouter</button>
    </form>
  )
}

const API_URL = 'http://127.0.0.1:8000/tasks'

function App() {
  /** @type {[Task[], import('react').Dispatch<import('react').SetStateAction<Task[]>>]} */
  const [tasks, setTasks] = useState([])
  const [remainingOnly, setRemainingOnly] = useState(false)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState('')
  const [actionError, setActionError] = useState('')

  useEffect(() => {
    let cancelled = false

    async function loadTasks() {
      try {
        const response = await fetch(API_URL)
        if (!response.ok) {
          throw new Error(`Erreur HTTP ${response.status}`)
        }
        const data = await response.json()
        if (!cancelled) setTasks(data)
      } catch (err) {
        if (!cancelled) {
          setError(
            err instanceof Error ? err.message : 'Erreur de chargement',
          )
        }
      } finally {
        if (!cancelled) setLoading(false)
      }
    }

    loadTasks()
    return () => {
      cancelled = true
    }
  }, [])

  /**
   * @param {string} url
```

Annexe du code final

src/App.jsx • partie 3 sur 4 • Copie du fichier relu

```
 98    * @param {RequestInit} [options]
 99    */
100   async function request(url, options) {
101     setActionError('')
102     try {
103       const response = await fetch(url, options)
104       if (!response.ok) {
105         throw new Error(`Erreur HTTP ${response.status}`)
106       }
107       return await response.json()
108     } catch (err) {
109       setActionError(err instanceof Error ? err.message : 'Erreur réseau')
110       return null
111     }
112   }
113
114   /** @param {string} title */
115   async function addTask(title) {
116     const created = await request(API_URL, {
117       method: 'POST',
118       headers: { 'Content-Type': 'application/json' },
119       body: JSON.stringify({ title }),
120     })
121     if (created) setTasks((currentTasks) => [...currentTasks, created])
122   }
123
124   /** @param {number} taskId */
125   async function toggleTask(taskId) {
126     const task = tasks.find((t) => t.id === taskId)
127     if (!task) return
128     const updated = await request(`${API_URL}/${taskId}`, {
129       method: 'PATCH',
130       headers: { 'Content-Type': 'application/json' },
131       body: JSON.stringify({ done: !task.done }),
132     })
133     if (updated) {
134       setTasks((currentTasks) =>
135         currentTasks.map((t) => (t.id === taskId ? updated : t)),
136       )
137     }
138   }
139
140   /** @param {number} taskId */
141   async function deleteTask(taskId) {
142     const result = await request(`${API_URL}/${taskId}`, { method: 'DELETE' })
143     if (result) {
144       setTasks((currentTasks) => currentTasks.filter((t) => t.id !== taskId))
145     }
146   }
```

Annexe du code final

src/App.jsx • partie 4 sur 4 • Copie du fichier relu

```
147   const visibleTasks = remainingOnly
148     ? tasks.filter((task) => !task.done)
149     : tasks
150
151   const remainingCount = tasks.filter((task) => !task.done).length
152
153   useEffect(() => {
154     document.title = `${remainingCount} tâche(s) restante(s)`
155   }, [remainingCount])
156
157   return (
158     <main>
159       <TaskHeader />
160       <TaskForm onAdd={addTask} />
161       <label>
162         <input
163           type="checkbox"
164           checked={remainingOnly}
165           onChange={(event) => setRemainingOnly(event.target.checked)}
166         />
167         Tâches restantes seulement
168       </label>
169       {actionError && <p role="alert">Erreur : {actionError}</p>}
170       {loading ? (
171         <p>Chargement...</p>
172       ) : error ? (
173         <p role="alert">Erreur : {error}</p>
174       ) : visibleTasks.length === 0 ? (
175         <p>Aucune tâche</p>
176       ) : (
177         <ul>
178           {visibleTasks.map((task) => (
179             <TaskItem
180               key={task.id}
181               task={task}
182               onToggle={toggleTask}
183               onDelete={deleteTask}
184             />
185           ))}
186         </ul>
187       )}
188     </main>
189   )
190 }
191
192 export default App
193
```

Annexe du code final

src/main.jsx • partie 1 sur 1 • Copie du fichier relu

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```